

Aurora Vine: An Integrated AI-Assisted System for Study Abroad Planning and Application Support

ISI Group 5

1 Project Idea

1.1 Problem Description

In the current study abroad application landscape, undergraduate students face multiple challenges when preparing for graduate applications. First, relevant information is highly fragmented, including university and program details, application cases, and experience sharing, which are distributed across heterogeneous platforms without unified integration. This fragmentation significantly increases the cost of information acquisition and makes systematic comparison and decision-making difficult [1].

Second, the application process is inherently long-term and multi-staged, involving preparation, positioning, material development, and submission. Students often lack clear guidance in decomposing high-level goals into executable steps, leading to inefficient planning and poor time allocation [2].

Finally, although various suggestions and reference experiences are available, there is a lack of effective mechanisms to transform these abstract recommendations into structured tasks and continuously track their execution. This results in a critical gap between planning and actual actions, limiting users' ability to achieve consistent progress [3].

1.2 Project Objectives

This project aims to develop a study abroad planning and application support system named **Aurora_Vine**, designed to provide integrated, end-to-end assistance for undergraduate applicants. The specific objectives are as follows:

- Build a unified information platform that integrates university and program data as well as application cases, thereby reducing the cost of information acquisition
- Provide user profiling and background modeling functions to structurally manage and present individual academic and experiential data
- Support the formulation and management of study plans by decomposing long-term goals into staged tasks
- Enable daily check-in and progress tracking to improve the consistency and controllability of plan execution
- Introduce an AI-assisted module based on external large model APIs to analyze user profiles and generate planning recommendations
- Transform AI-generated outputs into structured tasks, mapping planning content to executable system modules

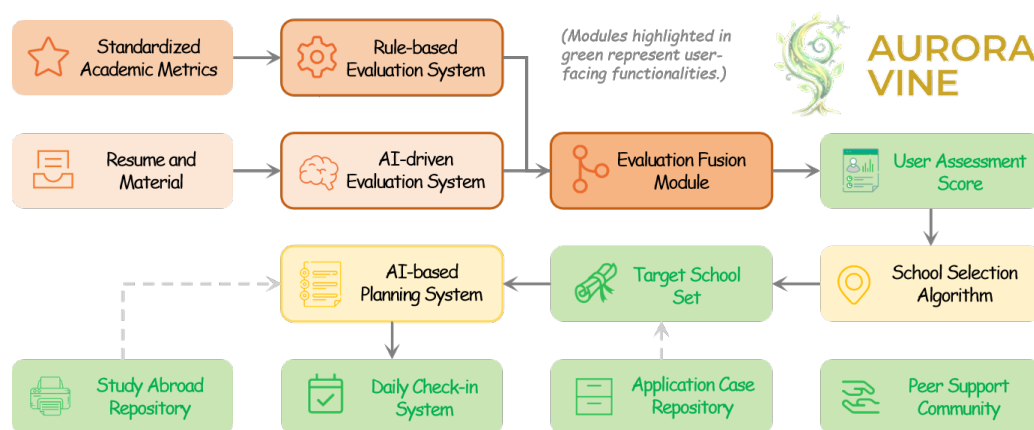


Figure 1. Overall system workflow of Aurora_Vine. The system follows an end-to-end pipeline from user input and evaluation to planning, execution, and feedback. Modules highlighted in green represent user-facing functionalities.

1.3 Project Relevance

As illustrated in Figure 1, Aurora Vine organizes the study abroad application process into a structured pipeline that connects information acquisition, user evaluation, planning generation, and execution tracking. By integrating these components within a unified system, it enables the transformation from fragmented information and abstract recommendations into coherent and actionable workflows.

For undergraduate students, study abroad application is a critical and complex decision-making process that directly influences their academic development and career trajectory. However, existing tools are often either information-centric or functionally isolated, lacking systematic support across the entire lifecycle.

Aurora.Vine addresses this gap by combining information aggregation, structured planning, AI-assisted analysis, and execution monitoring into a single framework. This design improves users' decision-making efficiency while enhancing their ability to consistently execute long-term plans, demonstrating strong practical applicability.

The system is currently under active development, and its implementation is publicly available at: https://github.com/TheBlueBanisters/Aurora_Vine

2 Target Users and Needs

2.1 Target Users

The primary target users of this project are undergraduate students, particularly those planning to apply for graduate programs domestically or internationally. Within the Software Development Life Cycle (SDLC), this stage corresponds to the phases of *Planning and Requirement Analysis* and *Defining Requirements*, where real user scenarios are analyzed to identify system problems and required functionalities.

As illustrated in Figure 2, the system primarily targets undergraduate students approaching graduation, representing the typical demographic and contextual scenario of users preparing for further academic progression.



Figure 2. Typical undergraduate students preparing for graduation and future academic progression. The image illustrates the primary user group targeted by the system.

Based on different stages of application preparation, target users can be categorized into two groups:

- **Exploration-stage users:** Students who have not yet determined their application direction and need to acquire foundational knowledge, including university information, program selection, and application difficulty.
- **Planning-stage users:** Students who already have preliminary goals and require background evaluation, application planning, and task execution management.

This classification helps clarify diverse usage scenarios during the requirement definition phase and guides subsequent functional design.

- **Information Acquisition:** The system should provide browsing, search, and filtering functions for universities, programs, and historical application cases, helping users build a fundamental understanding of the application landscape.
- **Profile Construction and Evaluation:** The system should allow users to input academic background and personal experiences, presenting them in a structured manner to support application positioning and subsequent planning.
- **Planning Generation:** The system should support the creation of application plans, including timeline arrangement, stage division, and task decomposition, enabling the transformation from abstract goals to actionable steps.
- **Execution Tracking:** The system should support daily check-ins and task status updates, enabling continuous tracking and feedback on plan execution.
- **AI-Assisted Functions:** The system should integrate external large-model APIs to analyze user profiles, generate evaluation and planning suggestions, and transform outputs into structured tasks integrated into modules such as planning and daily tracking.

2.2 Non-Functional Requirements

Non-functional requirements describe the quality constraints and operational characteristics that the system must satisfy beyond its functional capabilities.

- **Usability and Reliability:** The system should provide a clear and intuitive user interface, ensuring stable operation of core functions (e.g., data input, plan generation, and task tracking) while avoiding interruptions or data

inconsistencies.

- **Performance:** The system should achieve responsive performance in a local environment, ensuring smooth processes for information browsing, data processing, and plan visualization.
- **Maintainability and Compatibility:** The system should adopt a modular design to reduce coupling between components, facilitating future expansion and maintenance while ensuring stable compatibility across target environments.
- **Security and Controllability of AI Outputs:** The system should provide basic protection for user data. Additionally, due to the uncertainty of AI-generated outputs, results should be treated as assistive suggestions, allowing users to review, modify, and confirm them to ensure overall system controllability.

3 Project Scope

This section corresponds to the *Defining Requirements* phase in the Software Development Life Cycle (SDLC). At this stage, previously identified user needs are further transformed into concrete system functional modules, thereby clarifying the functional scope of the system. Through structured decomposition of functionalities, the system design can maintain clarity and consistency, while providing a solid foundation for subsequent system architecture design and implementation.

3.1 User and Data Management

This module is responsible for fundamental data management and user information maintenance, serving as the prerequisite for system operation. By constructing structured user profiles, the system provides essential data support for subsequent analysis and planning.

- **User Account Management:** Supports user registration, login, and basic information management, ensuring that users can access system functionalities with authenticated identities.
- **Profile Construction:** Allows users to input academic background and personal experiences (e.g., grades, project experience), storing and presenting them in a structured format to support downstream analysis and planning.

3.2 Information Acquisition and Support

This module provides fundamental informational support related to study abroad applications, serving as an important basis for user decision-making and planning.

- **University and Program Information Browsing:** Provides display and filtering functions for universities and programs, helping users understand different application options and their basic requirements.
- **Application Case Retrieval:** Provides access to historical application cases for reference and comparison, enabling users to evaluate their own profiles based on real-world examples.

3.3 AI-Assisted Analysis and Planning

This module is one of the core components of the system. By integrating AI capabilities through external large-model APIs, it enables the transformation from user data to actionable planning suggestions.

- **Profile-Based AI Analysis and Evaluation:** The system processes user background information to perform comprehensive analysis and generate preliminary evaluations of application competitiveness.
- **Study Plan Generation:** Based on analysis results, the system generates high-level application plans, including stage division, goal setting, and pathway design.
- **Plan Structuring:** Transforms AI-generated plans into structured tasks within the system, mapping them to subsequent planning management and execution modules, thereby bridging the gap between recommendations and actions.

3.4 Planning Execution and Tracking

This module is responsible for converting plans into continuous execution processes, while recording and providing feedback on execution status to form a complete closed loop.

- **Plan Management:** Allows users to view, adjust, and manage overall plans, ensuring that plans can be updated dynamically according to real-world conditions.
- **Daily Check-in and Progress Tracking:** Supports recording of daily task completion and continuous tracking of execution progress, thereby improving the consistency and controllability of plan execution.

3.5 Explicit Exclusions

To ensure feasibility and maintain a clear project boundary, several aspects are explicitly excluded from the scope of this project:

- **No in-house model training:** The project does not include the design or training of proprietary large language models. AI-related capabilities are limited to the integration of external model APIs as assistive components.
- **No end-to-end application submission automation:** The system is not intended to automatically complete official application submission processes, interact with university portals, or replace external third-party application systems.
- **No replacement of human decision-making:** The system provides planning support and reference suggestions, but it does not function as a substitute for professional academic advisors, institutional policies, or final admission decisions.
- **No full multi-platform expansion in this stage:** The current project scope focuses on a desktop-based local system and does not cover a full-scale mobile or web deployment.

4 Technical Specification

This section corresponds to the *Designing the Product Architecture* phase in the Software Development Life Cycle (SDLC). At this stage, the technical implementation approach and overall system architecture are defined to support the functional modules described previously. This section focuses on the system platform, technology stack, and the implementation of the AI module.

4.1 System Platform

Aurora_Vine is developed as a desktop application based on the **Electron** framework. This architecture enables the use of web technologies (HTML, CSS, and JavaScript) to build cross-platform desktop applications, while leveraging Node.js to provide system-level capabilities.

Compared to traditional web applications, the desktop-based approach better supports local data storage and offline usage scenarios, which aligns with the need for persistent data recording and management throughout the study planning process.

```

1 const OpenAI = require("openai");
2
3 const client = new OpenAI({
4   apiKey: "$MOONSHOT_API_KEY",
5   baseURL: "https://api.moonshot.cn/v1",
6 });
7
8 async function main() {
9   const completion = await client.chat.completions.create({
10     model: "kimi-k2.5",
11     messages: [
12       {
13         role: "system",
14         content: "You are Kimi, an AI assistant provided by Moonshot AI. You are proficient in both Chinese and English conversations. You provide safe, helpful, and accurate responses. You must refuse to answer any requests related to terrorism, racial discrimination, or explicit violent or inappropriate content. 'Moonshot AI' is a proprietary name and must not be translated into other languages."
15       },
16       {
17         role: "user",
18         content: "Hello, my name is Li Lei. What is 1 + 1?"
19       }
20     ]
21   });
22
23   console.log(completion.choices[0].message.content);
24 }
25
26 main();

```

Listing 1. Example of Kimi API integration using the Moonshot platform

4.2 Technology Stack

The system adopts a lightweight technology stack to ensure development efficiency and maintainability:

- **Frontend:** HTML, CSS, and JavaScript are used to construct the user interface, supporting information display and interactive logic.
- **Backend:** Node.js is used to handle system logic and data processing, communicating with the frontend through

interfaces provided by Electron.

- **Database:** SQLite is adopted as a local database to store user information, university data, application cases, and planning tasks.

This combination supports modular development while satisfying the requirements for local deployment and data management.

4.3 AI Module Implementation

The AI module is a key component of the system, enabling intelligent assistance through external large-model APIs. The system supports integration with multiple model services (e.g., DeepSeek [4], Kimi [5], Hunyuan [6]) to improve flexibility and adaptability.

The implementation follows a standard input–processing–output pipeline:

- **Input:** User-provided background information, including academic performance, project experience, and other relevant data.
- **Processing:** User data is packaged into structured requests and sent to external model APIs for analysis and generation.
- **Output:** The system generates study plan recommendations, which are further transformed into structured tasks and written into the planning and tracking modules.

This design integrates AI capabilities into the system in a modular manner, enabling intelligent assistance in the planning process without introducing the overhead of complex model training.

To demonstrate the feasibility of integrating external large-model APIs, a simplified example using the Kimi API is shown in Listing 1 [5]. The example illustrates how user input can be sent to the model and how responses are retrieved and processed within the system.

4.4 System Prototype and Interface Preview

The system is currently under active development. Figure 3 presents a prototype interface of Aurora Vine, which is designed to collect and structure user background information for subsequent planning and analysis.

The screenshot shows the 'AURORA VINE' logo in the top left. The sidebar on the left lists various navigation options under different categories. The main panel is titled 'School Planning' and contains a welcome message, a 'Basic Info' section with several input fields and dropdown menus, and a 'Language & Test Scores' section at the bottom.

Figure 3. Prototype interface of Aurora Vine for user information collection and planning (in development).

5 Development Methodology

5.1 Method Selection

This project adopts **Agile Development** as the overall development methodology. In addition, considering the uncertainty introduced by the AI module, the project incorporates **Dual Track Agile** as a complementary strategy to better coordinate AI exploration and system implementation.

5.2 Method Overview

5.2.1 Agile Development

Agile Development is an iterative approach that emphasizes adaptability and continuous delivery in dynamic development environments. Its core philosophy is captured in the **Agile Manifesto**:

*Individuals and interactions over processes and tools.
Working software over comprehensive documentation.
Customer collaboration over contract negotiation.
Responding to change over following a plan.*

These principles guide the development process by prioritizing flexibility, user-centered iteration, and continuous improvement, which are essential for handling evolving requirements in this project.

5.2.2 Dual Track Agile

Dual Track Agile extends traditional agile methods to address the mismatch between AI development and conventional software development cycles.

Its core components include:

- **Two Parallel Tracks:**
 - **Discovery Track:** Focuses on experimentation and optimization of AI-related functionalities.
 - **Delivery Track:** Focuses on system development and implementation.
- **Separation of Concerns:** Decouples AI exploration from system development to prevent uncertainty from affecting overall progress.
- **Placeholder APIs / Mocking:** Uses simulated interfaces to support system development before AI modules are fully implemented, enabling parallel progress.
- **Progressive Integration:** Gradually integrates AI capabilities into the system once they become stable.

6 Timeline and Arrangement

This project follows an agile development methodology. Development progresses iteratively through defined stages, with each phase centered around delivering a functional system increment and achieving key milestones. Meanwhile, team responsibilities are assigned based on functional roles and individual expertise to ensure feasibility and effective collaboration throughout the development process.

6.1 Development Phases and Milestones

longtable

Table 1. Development phases and milestones of Aurora Vine.

Phase	Timeline	Main Tasks	Milestones
Phase 1: Requirement Analysis and Design	Weeks 1	Requirement analysis, functional decomposition, technical design, and development preparation	Clear system requirements and completion of overall system design
Phase 2: Core Function Development	Weeks 2–3	Implementation of core modules including user management and information acquisition, with parallel preliminary AI integration and testing under Dual Track Agile	Core functionalities completed with an initial working prototype
Phase 3: AI Module Integration	Weeks 4–5	Integration of external large-model APIs for AI analysis, planning generation, and task structuring, with iterative refinement of earlier parallel development	AI-assisted functionalities operational with basic testing completed

Phase	Timeline	Main Tasks	Milestones
Phase 4: System Integration and Testing	Weeks 6	System integration, optimization, testing, and final version preparation	Stable system operation and completion of final release version

6.2 Team Roles and Responsibilities

The project team consists of 10 members. Roles are assigned based on functional responsibilities and individual expertise, covering frontend development, backend development, database management, AI module integration, and system testing.

No.	Member	Responsibilities
1	SIYAN WU	Leads project planning, defines requirements, prioritizes features, and coordinates major system decisions
2	YIXIANG CHEN	Designs system architecture, organizes development tasks, and coordinates implementation across modules
3	CHENGHUA ZHU	Develops key functional modules and supports core integration and major technical decisions
4	JINGYUAN ZHU	Supports frontend development, interface refinement, and debugging of user interaction functions
5	JIARUI FENG	Supports backend implementation, system logic development, and debugging of core functions
6	YUNQI CHEN	Conducts literature review and technical research to support system design and AI integration
7	GUOMENG LAI	Conducts market research and analyzes existing study-abroad platforms and related tools
8	YITAO WANG	Collects and organizes data to support content construction and information management
9	PEIZHEN LIN	Assists with AI testing, prompt refinement, and evaluation of generated results
10	CHENCHEN MING	Supports system integration, functional testing, and debugging of the final version

References

- [1] J. Khanal and U. Gaulee, “Challenges of International Students from Pre-Departure to Post-Study: A Literature Review,” *Journal of International Students*, vol. 9, no. 2, 2019, doi: 10.32674/jis.v9i2.673.
- [2] K. H. Kim and H. Spencer-Oatey, “Enhancing the Recruitment of Postgraduate Researchers from Diverse Countries: Managing the Application Process,” *Higher Education*, vol. 82, no. 5, pp. 917–935, 2021, doi: 10.1007/s10734-021-00681-z.
- [3] G. Martins van Jaarsveld, L. Y. J. Wong, M. Baars, M. M. Specht, and F. Paas, “Goal Setting in Higher Education: How, Why, and When Are Students Prompted to Set Goals? A Systematic Review,” *Frontiers in Education*, vol. 9, 2025, doi: 10.3389/feduc.2024.1511605.
- [4] DeepSeek-AI, “DeepSeek-V3 Technical Report,” *arXiv preprint arXiv:2412.19437*, 2025.
- [5] Moonshot AI, “Quickstart with the Kimi API,” *Kimi API Platform*. [Online]. Available: <https://platform.moonshot.ai/docs/guide/start-using-kimi-api>. [Accessed: Mar. 24, 2026].
- [6] Tencent Cloud, “Tencent Hunyuan Large Model API Overview,” *Tencent Cloud Documentation*. [Online]. Available: <https://cloud.tencent.com/document/product/1729/101848>. [Accessed: Mar. 24, 2026].